42112 — Story Sheet

Every parameter, variable and constraint — in plain English with an example

§6.5 Micro Brewery 2 — Assignment 5.1

A brewer can only run one beer through the tank per month. Everything you don't sell sits in storage costing money.

Element	Story / Example
Demand[b,m]	Thirsty customers — how many litres they want this month *e.g. Demand[Stout, Jan] = 35 means customers want 35 L of Stout in January*
BrewCap = 120	The tank holds 120 litres max *e.g. You cannot brew 130 L of Dark in March, even if demand calls for it*
StoreCap = 300	The cellar holds 300 litres total *e.g. 150 L Stout + 100 L Dark + 60 L Light = 310 L — that violates the limit*
Cost = 0.1	Every litre sitting in the cellar costs 10 cents a month *e.g. 60 L left over at end of June = €6 storage bill for that month*
InitialStorage[b]	What's already in the cellar when you start January *e.g. InitialStorage[Stout]=25, so you begin with 25 L of Stout already stored*
x[b,m]	How much you brew this month *e.g. x[Dark, Apr]=90 means you brew 90 L of Dark in April*
y[b,m]	How full the cellar is at the end of the month *e.g. y[Stout, Jan]=40 means 40 L of Stout remain after January sales*
q[b,m]	Did the brewery run this month? Yes or no *e.g. q[Light, Jun]=1 means you chose to brew Light in June; q[Stout, Jun]=0*
Objective	Minimise the cellar bill — every litre stored costs money *e.g. A plan with avg 50 L in cellar each month costs 50×0.1×12 = €60*
Inventory balance	Cellar tonight = cellar last night + what you brewed – what you sold *e.g. y[Stout,Feb] = y[Stout,Jan] + x[Stout,Feb] – Demand[Stout,Feb] = 40+0–20 = 20*
$x \le BrewCap \cdot q$	Can't brew if the brewery isn't running; if it is, max 120 L *e.g. q[Dark,Mar]=0 forces x[Dark,Mar]=0. q=1 allows up to x=120*
$\sum q \le 1$	Only one tap on the tank — one beer type at a time *e.g. q[Stout,May]+q[Dark,May]+q[Light,May] ≤ 1: pick at most one*
$\sum y \le StoreCap$	The cellar only fits 300 litres total *e.g. y[Stout,Jul]+y[Dark,Jul]+y[Light,Jul] ≤ 300*

§6.9 TA Workplan — Assignments 9.1 & 9.2

Four teaching assistants staff a help desk. Students need help at specific times. TAs hate inconvenient shifts. Nobody should arrive, leave, come back.

Element	Story / Example
Demand[p,d]	How many TAs must be at the desk at this hour *e.g. Demand[9-10, Mon]=2 means two TAs must be present Monday 9-10*
Inconvenience[ta,p,d]	How much this particular TA hates this particular slot *e.g. Inconvenience[AL,9-10,Mon]=8 means AL really dislikes early Monday mornings*
x[ta,p,d]	Is this TA standing at the desk right now? *e.g. x[FR,14-15,Wed]=1 means FR works Wednesday 14-15*
y[ta,p,d]	Did this TA just walk in the door this slot? *e.g. y[JE,11-12,Tue]=1 means JE's shift starts at 11 on Tuesday*
Objective	Minimise total TA misery *e.g. If AL works 3 slots with inconvenience 8,5,3 — that's 16 added to the total*
$\sum x \geq$ Demand	Enough bodies at the desk every hour *e.g. If Demand[10-11,Thu]=2 then x[AL]+x[FR]+x[JE]+x[MI] ≥ 2 for that slot*
$\sum x = 52$	Each TA works their contracted hours — no more, no less *e.g. AL's total working slots across all 9 days and 8 periods must equal exactly 52*
$\sum y \leq 1$	You can only arrive once per day *e.g. FR can't arrive at 10:00 and again at 14:00 on the same day*
$x \leq x\_prev + y$	You're only here if you were here last hour, or you just arrived *e.g. x[MI,13-14,Fri] ≤ x[MI,12-13,Fri] + y[MI,13-14,Fri]: MI works 13-14 only if already there or just arrived*
$\sum x \geq 2 \cdot \sum y$	If you bothered coming in, you stay at least 2 hours *e.g. JE arrives once ($\sum y=1$), so $\sum x \geq 2$: JE works at least 2 slots that day*

§6.12 Tennis — Assignments 12.1 – 12.4

A court sweeper must drag a brush over every line. They can walk between lines freely but want to minimise total footsteps.

Element	Story / Example
Distance[p1,p2]	How far apart two intersection points are *e.g. Distance[A0,B0] = 4.5 ft (they're 4.5 ft apart along the baseline)*
Lines[p1,p2]	Is there a line between these two points that needs sweeping? *e.g. Lines[B1,C1]=1 means the service line from B1 to C1 must be swept*
x[p1,p2]	Does the sweeper walk from point A to point B? *e.g. x[A0,B0]=1 means the sweeper walks from corner A0 to point B0*
s[p]	Did the sweeper start their shift here? *e.g. s[A0]=1, all others 0: the sweeper begins at corner A0*
e[p]	Did the sweeper finish their shift here? *e.g. e[E2]=1: the sweeper ends at the far corner E2*
Objective	Minimise total steps taken *e.g. If the optimal route is 280 ft, any other valid route is ≥ 280 ft*
Flow conservation	Every time you walk into a point, you also walk out — you can't teleport *e.g. The sweeper enters B1 twice (from B0 and from C1) and leaves B1 twice*
$\sum s = 1$, $\sum e = 1$	One place to start, one place to finish *e.g. Exactly one s[p]=1 and one e[p]=1 across all 12 points*
x[p1,p2]+x[p2,p1] ≥ Lines	Every line gets swept — in at least one direction *e.g. Lines[B1,C1]=1, so x[B1,C1]+x[C1,B1] ≥ 1: that line is swept left-to-right or right-to-left*
fix(s/e=0 interior)	Can't start or end mid-court — you'd walk back across clean clay *e.g. s[C1]=0 and e[C1]=0: the centre service-line junction is forbidden as start/end*

§6.14 Tariff Rates — Assignments 14.1 & 14.2

A power company decides each night how many generators to run across five time blocks. Starting a generator costs a one-time fee. The plan repeats daily — yesterday's last block is today's first block's predecessor.

Element	Story / Example
PDEM[h]	How hungry the grid is in this time block *e.g. PDEM[4]=40,000 MW — the afternoon peak is the hardest period to cover*
NHP[h]	How many hours this block lasts *e.g. NHP[2]=3 (06:00-09:00), so costs in period 2 are multiplied by 3*
NG[p]	How many generators of this type the company owns *e.g. NG[1]=12: you can run at most 12 type-1 generators at once*
MINL / MAXL[p]	Every generator has a minimum it must run at, and a ceiling it can't exceed *e.g. A type-1 generator must produce between 850 MW and 2,000 MW — it can't idle at 100 MW*
MINC[p]	Cost per hour just to keep one generator ticking at minimum *e.g. One type-2 generator costs €2,600/h just to keep on, before any extra production*
PC[p]	Extra cost per MW above minimum — ramping up is expensive *e.g. Type-1 costs €2 per extra MW·h; pushing from 850 to 1,000 MW costs 150×2=€300/h extra*
STARTC[p]	The ignition fee — charged every time a cold generator starts *e.g. Starting one type-1 generator costs €2,000 — do it twice and that's €4,000*
nRun[p,h]	How many of this generator type are humming right now *e.g. nRun[1,4]=8 means 8 type-1 generators are running during the afternoon peak*
prod[p,h]	Total MW this generator type is pushing to the grid *e.g. prod[1,4]=12,000 means type-1 generators collectively produce 12,000 MW in period 4*
nStart[p,h]	How many generators of this type just switched on this block *e.g. nRun went from 5 to 8 between periods 1 and 2, so nStart[1,2] ≥ 3*
Objective	Pay the running bill + the ramping bill + the ignition bill *e.g. Period 2 (3h): 8 type-1 at min = 8×1000×3=€24,000 running cost, plus ramping, plus any startups*
prod ≥ MINL · nRun	A running generator can't idle below its minimum — physics *e.g. 3 type-1 generators running → prod[1,h] ≥ 3×850 = 2,550 MW*
prod ≤ MAXL · nRun	A running generator can't exceed its rated maximum — also physics *e.g. Those same 3 generators → prod[1,h] ≤ 3×2,000 = 6,000 MW*
∑prod ≥ PDEM	The lights stay on — demand is always met *e.g. In period 4: prod[1,4]+prod[2,4]+prod[3,4] ≥ 40,000 MW*
nStart ≥ nRun−nRun_prev	More running now than last block? Someone pressed start *e.g. nRun[2,2]=8, nRun[2,1]=5 → nStart[2,2] ≥ 8−5=3 type-2 generators started up*
∑MAXL·nRun ≥ 1.15·PDEM	Even if demand spikes 15%, enough generators are warm to cover it *e.g. Period 4 reserve: running generators must be able to produce ≥ 1.15×40,000=46,000 MW if called upon*

§6.17 Wedding Planner — Assignments 17.1 – 17.5

A wedding planner seats 20 guests across 3 tables. Couples must stay together. Beyond that: guests who share interests sit close, genders balance, nobody is surrounded by strangers.

Element	Story / Example
TableCap = 9	No table can be crowded beyond 9 seats *e.g. If 10 guests are assigned to table 2, the constraint is violated*
Couple[g1,g2]	These two come as a package deal *e.g. Couple[3,4]=1 means guests 3 and 4 must sit at the same table*
SharedInterests[g1,g2]	How much these two have to talk about *e.g. SharedInterests[5,12]=4 means guests 5 and 12 share 4 interests — seat them together if possible*
Male[g] / Female[g]	Guest g's gender flag *e.g. Male[7]=1, Female[7]=0 means guest 7 is male*
Know[g1,g2]	Do these two already know each other? *e.g. Know[2,9]=1: guests 2 and 9 are acquainted; Know[2,15]=0: strangers*
x[g,t]	Is this guest sitting at this table? *e.g. x[7,2]=1 means guest 7 sits at table 2; x[7,1]=x[7,3]=0*
y[g1,g2,t]	Are these two guests sitting at the same table? *e.g. y[5,12,3]=1 means guests 5 and 12 are both at table 3 — their 4 shared interests are counted*
m[t] / f[t]	Excess men / women at this table beyond the allowed imbalance of 2 *e.g. Table 1 has 6 men, 2 women: 6–2=4 > 2, so m[1]=2 and €2 is added to the objective*
k[g]	Familiar faces this guest is missing — shortfall below 3 known people *e.g. Guest 5 is at table 1 but knows only 1 person there: k[5] ≥ 3–1=2*
Objective 17.2	Pack each table with people who have things in common *e.g. Placing guests 5 and 12 together contributes +4 to the objective*
Objective 17.3	Minimise gender imbalance across tables *e.g. m[1]+f[1]+m[2]+f[2]+m[3]+f[3]=0 means perfect balance at all tables*
Objective 17.4	Minimise guests who feel like strangers *e.g. If 3 guests each know only 1 person at their table, total shortfall = 3×2=6*
Objective 17.5	Do all three at once: –shared interests + gender slack + knowing slack *e.g. Objective = –64 + 2 + 2 = –60: interests dominate, small penalties for gender and knowing*
$\sum x = 1$	Every guest sits somewhere — exactly once *e.g. x[7,1]+x[7,2]+x[7,3]=1: guest 7 is at exactly one table*
$\sum x \le$ TableCap	Don't overcrowd the table *e.g. x[1,2]+x[2,2]+…+x[20,2] ≤ 9: table 2 holds at most 9 guests*
x[g1,t] = x[g2,t]	Couples are glued together *e.g. Couple[3,4]=1 → x[3,1]=x[4,1], x[3,2]=x[4,2], x[3,3]=x[4,3]*
y ≤ x[g1,t]; y ≤ x[g2,t]	Count a pair as 'together' only if both are actually at that table *e.g. y[5,12,3] ≤ x[5,3] and y[5,12,3] ≤ x[12,3]: if either isn't at table 3, y must be 0*
Male·x–Female·x ≤ 2+m[t]	At most 2 more men than women — or pay a penalty in m[t] *e.g. Table 1: 6 men, 2 women → 6–2=4 > 2, so m[1] must be at least 2*
k–3·x+$\sum$Know·y ≥ 0	Know at least 3 people at your table — or the shortfall lands in k[g] *e.g. Guest 5 at table 1, knows 1 person there: k[5]–3×1+1 ≥ 0 → k[5] ≥ 2*

SIP Production Planning — Pre-oral Submission

A 3D printing shop picks which customer orders to run today. Three machines all have 8-hour limits. In Assignment 2, delivering to a buyer costs a flat fee — one van per buyer, regardless of how many items.

Element	Story / Example
Price[i]	What the customer offered to pay *e.g. Price[1]=397 means item 1 earns €397 if produced*
PrintTime[i]	How long the 3D printer needs for this item *e.g. PrintTime[1]=49 means item 1 ties up the printer for 49 minutes*
PolishTime[i]	How long the polishing machine needs *e.g. PolishTime[1]=10: item 1 is quick to polish*
PaintTime[i]	How long the painting machine needs *e.g. PaintTime[1]=17: the painter spends 17 min on item 1*
Minutes = 480	Each machine runs for 8 hours — then it stops *e.g. 8 hours × 60 min = 480 min available per machine per day*
BuyerId[i]	Whose order is this item part of? *e.g. BuyerId[1]=12 means item 1 belongs to buyer 12*
TransportCost = 50	One delivery van per buyer, regardless of how many items *e.g. Producing items 1 and 3 (both buyer 12) costs one delivery: 50 DKK total, not 100*
x[i]	Do we produce this item today? *e.g. x[7]=1 means item 7 is selected for production; x[7]=0 means it is skipped*
y[b]	Are we sending a delivery to this buyer today? *e.g. y[12]=1 means buyer 12 gets a delivery — and we pay the 50 DKK transport cost*
Objective 1	Maximise today's revenue *e.g. Select items 1,5,15: revenue = 397+230+399 = €1,026 (before machine limits apply)*
Objective 2	Maximise revenue minus delivery costs *e.g. Revenue €7,850 – 8 buyers × 50 DKK = net 7,450 (mixed units)*
$\sum$PrintTime·x $\leq$ 480	The printer can't run beyond its shift *e.g. Items 1+2+3 use 49+10+45=104 print-min; keep adding until you approach 480*
$\sum$PolishTime·x $\leq$ 480	The polisher can't run beyond its shift *e.g. Polish is rarely the binding constraint — the printer and painter fill up first*
$\sum$PaintTime·x $\leq$ 480	The painter can't run beyond its shift *e.g. The optimal solution uses 479/480 print-min and 479/480 paint-min — both nearly full*
x[i] $\leq$ y[BuyerId[i]]	Can't produce an item without sending a delivery to its buyer *e.g. x[1]=1 (item 1, buyer 12) forces y[12]=1, triggering the 50 DKK delivery cost*